

There are two primary "types" or approaches to using secret keys in JWT: **Symmetric** and **Asymmetric**.

1. Symmetric Encryption (Shared Secret)

This is the most common and simplest method. Both the party **issuing** the token and the party **verifying** it use the exact same string (the "secret").

- **Algorithm:** Usually **HS256** (HMAC with SHA-256).
- **Key Type:** A long, random string of bytes.
- **Best For:** Internal services or "monolithic" apps where the same server both creates and validates the token.
- **The Risk:** If an attacker gets this secret, they can forge tokens that your server will accept as 100% legitimate.

2. Asymmetric Encryption (Public/Private Key)

This method uses a mathematical pair: a **Private Key** to sign the token and a **Public Key** to verify it.

- **Algorithm:** Usually **RS256** (RSA) or **ES256** (Elliptic Curve).
- **How it works:** * The **Private Key** is kept strictly on the Identity Server to create tokens.
 - The **Public Key** is shared with any service that needs to check if the token is valid.
- **Best For:** Microservices or Third-Party APIs. A resource server (like a "Billing API") can verify the user's identity without ever knowing the private key used to sign the token.
- **The Benefit:** If the Public Key is leaked, no one can create new tokens; they can only verify existing ones.

Comparison Table

Feature	Symmetric (HS256)	Asymmetric (RS256)
Key Type	Single string (Secret)	Key Pair (Public/Private)
Performance	Faster	Slightly slower

Security	Lower (Shared secret)	Higher (Separation of duties)
Complexity	Easy to implement	Requires key management (PEM/JWKS)

Applied coding

Appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "Jwt": {
    "Issuer": "https://localhost:7030",
    "Audience": "https://localhost:7030",
    "Key": "A_Very_Long_And_Secure_Secret_Key_12345"
  }
}
```

Program.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;

using Microsoft.IdentityModel.Tokens;

using Microsoft.AspNetCore.Builder;

using System.IdentityModel.Tokens.Jwt;

using System.Security.Claims;

using System.Text;

using Microsoft.OpenApi.Models; // Added for cleaner Swagger types


var builder = WebApplication.CreateBuilder(args);


// 1. Setup Authentication
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,

            ValidateAudience = true,

            ValidateLifetime = true,

            ValidateIssuerSigningKey = true,

            ValidIssuer = builder.Configuration["Jwt:Issuer"],
```

```

        ValidAudience = builder.Configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
    };
});

builder.Services.AddAuthorization();

// 2. Setup Swagger with JWT Support
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "My API", Version = "v1" });

    // Define the "Bearer" security scheme
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.ApiKey,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "Enter 'Bearer' [space] and then your token.\r\n\r\nExample: \"Bearer 12345abcdef\""
    });
});

```

```

// Make sure Swagger actually sends the token in requests
c.AddSecurityRequirement(new OpenApiSecurityRequirement
{
    {
        new OpenApiSecurityScheme
        {
            Reference = new OpenApiReference
            {
                Type = ReferenceType.SecurityScheme,
                Id = "Bearer"
            }
        },
        Array.Empty<string>()
    }
});

});

var app = builder.Build();

// 3. Enable Swagger UI
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(); // This creates the interactive /swagger page
}

```

```
app.UseHttpsRedirection();
```

```
// 4. Order Matters!
```

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

```
// 5. Endpoints
```

```
app.MapGet("/", () => "Hello World!");
```

```
app.MapGet("/ProtectedInfo", () => "You are authenticated!")
```

```
.RequireAuthorization();
```

```
app.MapGet("/RolebasedInfo", () => "Hello Student!")
```

```
.RequireAuthorization(policy => policy.RequireRole("student"));
```

```
app.MapPost("/login", (UserLogin user) => {
```

```
    // Note: Use a strong key in appsettings.json!
```

```
    if (user.Username == "admin" && user.Password == "password")
```

```
    {
```

```
        var claims = new[] {
```

```
            new Claim(JwtRegisteredClaimNames.Sub, user.Username),
```

```
            new Claim(ClaimTypes.Role, "student") // Changed to 'student' to match your endpoint
```

```
        };
```

```
        var key = new
```

```
SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!));
```

```
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

var token = new JwtSecurityToken(
    issuer: builder.Configuration["Jwt:Issuer"],
    audience: builder.Configuration["Jwt:Audience"],
    claims: claims,
    expires: DateTime.Now.AddHours(1),
    signingCredentials: creds
);

return Results.Ok(new { token = new JwtSecurityTokenHandler().WriteToken(token) });
}
return Results.Unauthorized();
});

app.Run();

public record UserLogin(string Username, string Password);
```